Below is a “how to follow the checklist” instruction doc you can hand straight to Claude/Copilot. It includes both the design brief and the explicit checklist, plus guidance on how to use each part when coding.

All you need to do is paste this into your repo (e.g. SPEC.md) and tell Copilot: “Implement according to SPEC.md”.

SPEC: Self-Healing FTMO RL Trading System (Instructions + Checklist)

0. How to use this document

You (Claude/Copilot) must:

1. Read this spec top-to-bottom.
2. Treat it as the **source of truth** for:
    - What the system should do.
    - What constraints must be enforced.
3. Use the **checklists** as acceptance criteria:
    - Do not invent extra trading rules.
    - Do not simplify the FTMO logic.
4. Choose your own implementation details (code style, libraries, internal abstractions), as long as all items in the checklists are satisfied.

---

1. Design brief (high-level instructions)

1.1 Overall goal

Extend the existing Quantra RL template (Game, ExperienceReplay, run, etc.) into a **self-healing, FTMO-focused RL trading system** that:

- Trades **multiple MT5 instruments** (EURUSD, GBPUSD, XAUUSD, US30, NAS100) on **1D, 1H, 15m, 1m** timeframes.
- Learns a **general policy** that works across assets and timeframes.
- Lets the RL agent choose:
    - Direction (buy/sell/flat).

    - Lot size (small/med/large) relative to account size.
- Trains through a **4-phase curriculum** with strict masking rules:
    - Phases 1–3: rule-based regimes where the agent is allowed or required to trade only under specific indicator conditions.
    - Phase 4: full FTMO environment with no indicator masks.
- Optimizes to pass a **custom FTMO profile**:
    - Daily profit target: +2.5% of starting equity.
    - Trailing daily max drawdown: −1% from intraday equity high (including open trades).edgeflo+1
- Supports:
    - Distributed training (e.g., Spark/Ray) for many experiments in parallel.
    - A dashboard and logging for FTMO metrics.
    - SHAP-based explanation and chatbot hooks.
    - Future meta-learning on top of the base RL agent.

You are responsible for all code decisions, as long as you implement the interfaces and behavior described below.

---

2. Checklist for data & MT5

When coding:

1. Implement an mt5_bridge module that can:
    - Connect to MT5, login to the account (demo by default).
    - Load historical candles for:
        - Symbols: ["EURUSD", "GBPUSD", "XAUUSD", "US30", "NAS100"].
        - Timeframes: 1D, 1H, 15m, 1m.
2. Use these date ranges:
    - Training: 2018-01-01 → 2022-12-31
    - Validation: 2023-01-01 → 2023-12-31

    - Forward test: 2024-01-01 → present
3. Convert MT5 server time to **CET/CE(S)T**:
    - All “day” boundaries, equity resets, and time features (hour, day_of_week) must be defined in CET/CE(S)T.
4. Ensure data feeds into the RL environment in a consistent format (e.g., per-asset dict of pandas DataFrames or arrays).

---

3. Checklist for indicators & state features

You must build a **state vector** that includes the following for each asset and each timeframe (1D, 1H, 15m, 1m):

3.1 Price & volatility

- open, high, low, close, volume
- ATR(14)
- SMA(1), shift +2, applied to ATR(14)

3.2 Indicators from Example strategies (on all TFs)

Implement exactly these indicators with the given periods and shifts:

- STRAT-001
    - BB(200), shift 0
    - BB(20), shift 0
    - RSI(7), no shift
- STRAT-002
    - CCI(30), no shift
    - CCI(100), no shift
    - RSI(7), no shift
- STRAT-003
    - CCI(14), CCI(100), CCI(900), all no shift
    - SMA(20), shift 0, applied to each CCI

- STRAT-004
    - SMA(50), shift 0
    - SMA(4), shift 0
    - SMA(4), shift +4
- STRAT-005
    - SMA(30), shift 0
    - SMA(50), shift 0
    - RSI(5), no shift
- STRAT-006
    - ADX(14), SMA(1), shift +5 applied to ADX(14)
    - ATR(14), SMA(1), shift +5 applied to ATR(14)
- STRAT-007
    - SMA(50), shift 0
    - SMA(4), shifts 0, +1, +2, +3, +4
- STRAT-008
    - CCI(30), CCI(100), CCI(300), no shift
    - BB(14, dev 1, shift 0) on each CCI
- STRAT-009
    - SMA(200) on 5-minute chart, shift 0 (adapt as needed).
- STRAT-010
    - SMA(200), shift 0
- STRAT-011
    - CCI(140), no shift
    - CCI(14), no shift
    - SMA(1), shift +4 applied to CCI(140)

3.3 Extra indicators (all TFs)

- ATR(14), SMA(1), shift +2 applied to ATR(14).
- CCI(30), CCI(100), and SMA(1), shift +2 applied to each.
- Bollinger upper bands of BB(20) and BB(200), and:
    - SMA(4), shift +8, applied to each upper band.

3.4 Time & calendar (CET/CE(S)T)

- sin_hour, cos_hour from hour_of_day.
- sin_dow, cos_dow from day_of_week.

3.5 Position & account features

Per asset:

- position_side $\in \{-1, 0, +1\}$
- position_size_pct_equity
- unrealized_pnl_pct

Account-level:

- equity_pct_change_today
- current_daily_drawdown_pct
- days_in_current_streak (consecutive FTMO pass days)

You design the exact tensor layout, but all above information must be available to the model.

---

4. Checklist for action space

For each asset, at each 1m decision step, the agent must select one of:

- flat
- buy_small
- buy_med
- buy_large
- sell_small

- sell_med
- sell_large

Guidelines:

- Map these to position sizes as **learnable parameters** (not hard-coded risk fractions).
- You may initialize them sensibly (e.g., small/med/large as increasing risk), but allow them to be adapted by training or meta-logic, subject to risk constraints.

The network architecture (shared vs per-asset heads) is up to you, but it must support **multi-asset simultaneous actions**.

---

5. FTMO profile and constraints

Implement a **FTMO mode** with:

- daily_profit_target_pct = 0.025 (2.5%).
- daily_max_drawdown_pct = 0.01 (1%).academy.ftmo+1

Environment requirements:

- Track for each CET day:
    - start_equity_today.
    - intraday_high_equity.
- Enforce:
    - If equity <= intraday_high_equity * (1 − 0.01):
        - Stop opening new trades; mark day as **fail**.
    - If equity >= start_equity_today * (1 + 0.025):
        - Mark day as **target hit**; further trading must **not** allow DD breach.

Add TRADING_MODE: "ftmo" | "free" in a config:

- In "ftmo":
    - Use FTMO rules and rewards.
- In "free":

- Use PnL / risk-adjusted rewards without hard daily stop.

No fixed limits on number of positions or exposure, beyond basic safety (e.g., broker leverage), so the RL can discover its own allocation.

---

6. Checklist for curriculum phases (masking logic)

Implement 4 phases; advance when **10 consecutive pass days** occur in that phase (pass day = ≥2.5% return, no 1% DD breach).

Phase 1 – CCI alignment (1m + 15m)

Indicators per asset:

- 1m and 15m:
  - CCI(30), SMA(1, shift +2) on CCI(30).
  - CCI(100), SMA(1, shift +2) on CCI(100).

Condition to **allow new trades**:

- On 1m:
  - Either all above:
    - CCI30_1m > SMA1_shift2_CCI30_1m AND CCI100_1m > SMA1_shift2_CCI100_1m
  - Or all below:
    - CCI30_1m < SMA1_shift2_CCI30_1m AND CCI100_1m < SMA1_shift2_CCI100_1m

AND on 15m the same pattern (both above or both below).

Rules:

- New trades only allowed when this condition is true on both TFs.
- Agent chooses buy/sell and size.
- When condition is false:
  - No new trades allowed.
  - Existing trades can remain; agent must learn when to close.

Phase 2 – SMA on Bollinger upper band (1m + 1H)

For each asset and TF (1m, 1H):

- Upper band of BB(20) or BB(200) (choose consistently).
- SMA(4), shift +8, on that upper band.

Condition:

- On 1m:
    - price_1m > SMA_upper4_shift8_1m OR price_1m < SMA_upper4_shift8_1m.
- On 1H:
    - price_1H > SMA_upper4_shift8_1H OR price_1H < SMA_upper4_shift8_1H.

Direction must match:

- Both above, or both below.

Rules:

- When condition holds (same direction on both TFs), the agent **must be in an active trade** on that asset:
    - If flat, must enter (direction and size chosen by agent).
- When condition false:
    - Agent can be flat; no "must be in" requirement.

Phase 3 – Bollinger midlines (1m + 15m)

For each asset and TF (1m, 15m):

- Use middle lines of BB(200) and BB(20), shift 0.

Condition:

- On 1m:
    - Either:
        - price_1m > BB200_mid_1m AND price_1m > BB20_mid_1m
    - Or:
        - price_1m < BB200_mid_1m AND price_1m < BB20_mid_1m

- On 15m:
    - Same “above both” or “below both” condition.

Rules:

- When true on both TFs:
    - Agent must be in a trade.
- When false:
    - Flat allowed.

Phase 4 – FTMO consistency

- No indicator gating.
- Agent free to trade at any time, subject only to FTMO profile rules.
- Objective: maximize number of pass days (2.5% target, no 1% DD) and long streaks of such days.

---

7. Checklist for rewards (guidelines)

You design the exact formulas, but must include:

1. **Trade-level reward**:
    - Use existing Quantra PnL logic (e.g., exponential PnL) on closed trades.Reward-System.ipynb
2. **Daily-level reward**:
    - At end of each CET day, compute:
        - Daily return r_d.
        - Daily max drawdown dd_d.
    - Define:
        - Pass day: r_d ≥ 2.5% and dd_d ≤ 1%.
        - OK day: 0 ≤ r_d < 2.5% and dd_d ≤ 1%.
        - Fail day: else.
    - Assign:

        - Positive reward for pass day (largest).
        - Smaller positive for OK.
        - Negative for fail.ftmo+3
3. **Streak bonus**:
    - Additional reward ∝ days_in_current_streak (longer streak → higher bonus).
4. **Low-DD bonus**:
    - Extra reward when dd_d is significantly below 1%.

---

8. Checklist for training, Spark, dashboard, explainability, meta

You must:

- Wrap training in functions that accept config paths and data paths.
- Support parallel/distributed runs (Spark/Ray or similar).
- Maintain separate train/validation/forward-test regimes.
- Implement a dashboard (e.g., Streamlit/Dash) showing:
    - Equity curves.
    - Daily returns and drawdowns.
    - FTMO metrics and streaks.
- Integrate W&B (or similar) logging for:
    - Hyperparameters, rewards, drawdowns, pass/fail days.
- Implement a SHAP-based explain_action(state) for the final policy:
    - To answer "why did it trade like this?" for chatbot or dashboard.

A simple meta-learning layer can be left as a placeholder, but with code hooks that can later reweight agents based on performance.

If you hand this spec to Claude/Copilot and say "follow this exactly," it has everything needed to implement the system without you writing any code.

Is there any *one* part of this you'd like to add acceptance tests for (for example, a specific CSV output format for daily FTMO metrics) before you finalize this spec?

---

confirmation
Here's a first pass at what you asked for, broken into two parts:

1. A "design brief" document you can hand to Copilot (or paste at the top of a repo) to steer how it should extend your Quantra RL template into your self-healing FTMO-focused trading system.
2. A set of multiple-choice questions that will help you clarify details so this brief can become very precise.

---

1. Design brief for Copilot: "Self-Healing FTMO RL Trading System"

A. Overall goal

- Start from the Quantra RL template (Game class, ExperienceReplay, run function, etc.) and extend it into a **self-healing RL trading system** that can:
    - Trade FX (e.g., EURUSD) intraday using minute and resampled bars.
    - Be trained efficiently (eventually distributed using Spark or similar).
    - Optimize specifically for **passing FTMO-style challenges** under precise risk constraints.
    - Support **transfer / curriculum learning** to gradually relax constraints.
    - Allow an **FTMO "mode" toggle** vs "free-trade" mode.
    - Integrate with a **dashboard + chatbot + SHAP + Weights & Biases** to explain and monitor behavior.
    - Be extensible to **meta-learning** on top of the RL agent.

B. Use and extend the existing template

- Keep the core structure from Quantra's template:
    - Game class (state assembly, reward, act, reset).
    - ExperienceReplay and Bellman update.

    - run training loop and rlconfig dictionary.
    - State assembly (multi-timeframe OHLC, indicators, time signatures, position).
- Refactor into a **package-style structure**:
    - env/ for environment + FTMO logic.
    - agents/ for RL models, meta-learner, and curriculum/transfer tooling.
    - training/ for Spark/distributed training scripts and backtests.
    - monitoring/ for W&B, SHAP, logging, and dashboards.
    - config/ for YAML/JSON configs (FTMO mode, risk targets, daily objectives).

C. FTMO objective & risk constraints

1. **Primary objective**
    - Design reward(s) so that, in FTMO mode, the agent focuses on:
        - Achieving **at least 2.5% daily return** (or user-specified daily target).
        - Avoiding **more than 1% intraday drawdown** from daily equity peak.
    - Implement FTMO-specific rewards that combine:
        - PnL-based reward (like current rewardexponentialpnl) but scaled to the day's target.
        - Penalties when:
            - Equity falls more than allowed from the daily high.
            - Daily target is overshot with unnecessary risk.
            - Significant overnight or weekend exposure (if applicable).
2. **On/off FTMO mode**
    - Add a configuration flag: TRADING_MODE: "ftmo" | "free" in a config file.
    - In FTMO mode:
        - Use FTMO reward and risk constraints (daily target, max daily loss).
        - Enforce daily stop trading when constraints are hit.

- In free mode:
    - Use standard trading reward (Sharpe-like or pure PnL with risk penalty).
    - Remove FTMO-specific daily hard stops.

3. **Daily target / risk dashboard inputs**
    - Expose user inputs (via config or dashboard API) for:
        - daily_profit_target_percent (default 2.5%).
        - daily_max_drawdown_percent (default 1.0%).
    - Read these inputs inside the environment and reward logic, not hard-coded.

D. Curriculum / transfer-learning training framework

1. **Curriculum phases**
    - Implement training in **phases** (curriculum steps) like:
        - Phase 1: Very tight constraints, simple reward, small position sizes, maybe restricted hours.
        - Phase 2: Mildly relaxed constraints, introduce more indicators or longer holding periods.
        - Phase 3: Realistic FTMO constraints and full feature set.
    - Represent this with a curriculum_schedule config:
        - Each phase has: time-window of data, constraints, reward function, action space limits, allowed leverage.
2. **Transfer learning**
    - Reuse the Quantra PRELOAD weights mechanism and extend:
        - After finishing a phase, save the model weights and replay buffer.
        - Load them into the next phase with slightly modified environment/constraints.
    - Provide helpers to:

- Copy weights from a “base” agent to new agents (new symbol, new regime).
- Optionally freeze early layers and fine-tune last layers.

E. Training & forward testing setup (Spark, etc.)

1. **RL training on Spark / distributed**
    - Wrap the run(bars5m, rlconfig) function into a training job that can:
        - Run multiple seeds / configurations in parallel.
        - Possibly distribute episode batches across workers.
    - Provide a simple interface like:
        - spark_train(config_path, data_path, num_workers, num_episodes)
    - Ensure logging and model saving are compatible with distributed runs (unique paths per worker, W&B run IDs, etc.).
2. **Forward-testing pipeline**
    - Separate **train** / **validation** / **forward-test** date ranges.
    - After each training phase:
        - Automatically run a forward-test on unseen data (no training updates).
        - Log FTMO metrics: daily returns, daily drawdown, number of days to reach target, constraint violations.

F. Dashboard + chatbot + explainability

1. **Dashboard**
    - Build a separate monitoring/dashboard.py (or similar) that:
        - Shows daily PnL, daily drawdown, open positions, recent trades.
        - Lets the user input:
            - Daily PnL target.

        - Daily risk limit.
        - FTMO mode toggle.
    - Displays FTMO metrics over time (rolling 30-day stats, challenge-passing probability).

2. **Weights & Biases integration**
    - Add W&B logging hooks to RL training:
        - Hyperparameters, rewards, drawdown metrics, episodic returns.
        - FTMO-specific metrics (number of daily constraint hits, days meeting target).
    - Log model checkpoints and evaluation results for each curriculum phase.
3. **SHAP-based explanations + chatbot**
    - For the chosen RL model (e.g., Keras or PyTorch), expose:
        - A function explain_action(state) that:
            - Runs the model to get action logits/probabilities.
            - Uses SHAP (KernelExplainer or DeepExplainer) to compute feature contributions.
    - Provide an API for a chatbot that can answer:
        - "Why did the model go long/short/flat here?"
        - "Which features were most important today?"
        - "How has feature importance shifted over the last N trades?"
    - Use W&B or local storage to store SHAP summaries over time for the dashboard.

G. Meta-learning layer (future extension)

- Architect the system so later you can add a **meta-learner** that:
    - Observes performance of multiple agents or configurations (like Quantra's combining-agents notebook).

        - Adjusts hyperparameters, curriculum phase durations, or chooses among agents.
- Provide a simple meta-learner placeholder:
    - A module that can:
        - Read per-agent returns and drawdowns.
        - Reallocate capital across agents dynamically based on rolling performance.
    - Later, replace that with a proper meta-RL or bandit algorithm.

H. Checklist for "upgrading the bare-bones RL model"

Copilot should aim to satisfy this checklist when generating or modifying code:

1. **Environment & Game class**
    - FTMO mode flag recognized (TRADING_MODE: "ftmo" | "free").
    - Daily target and max drawdown parameters included and passed to Game.
    - Environment tracks daily equity, daily high, and daily drawdown.
    - Episode termination logic supports:
        - Trade-level termination (as in Quantra template).
        - Daily termination when FTMO constraints hit.
    - State remains multi-timeframe + indicators + time signature + position.
2. **Reward functions**
    - FTMO-specific reward functions added, e.g.:
        - Reward scaled by progress toward daily target.
        - Penalty for violating daily drawdown or coming close.
    - Existing PnL-based rewards kept for free mode.
    - Reward function chosen via rlconfig["RF"] or per-phase curriculum config.
3. **Curriculum / transfer learning**

- Curriculum schedule structure defined (list of phases with constraints, reward, data slices).
- Training loop can iterate through phases and save / reload model weights.
- Optional layer freezing / fine-tuning hooks for transfer learning.

4. **Training & Spark**
    - A script or module for distributed training using Spark or similar:
        - Accepts config, loads data, runs multiple workers.
    - Proper handling of model checkpoints, seeds, logging, so runs are reproducible.
5. **Forward testing**
    - Clean separation of train/validation/forward-test dates.
    - Automatic forward-test run after training with FTMO metrics.
    - Stored results (CSV + logs) for later analysis.
6. **Dashboard + config interface**
    - Config-driven inputs for daily target, risk, and mode.
    - Basic dashboard (web server or notebook-based) to view:
        - Equity curve.
        - FTMO metrics.
        - Current mode and targets.
7. **Explainability & monitoring**
    - W&B logging integrated into training.
    - SHAP explainer wired to the RL model's state → action mapping.
    - API endpoints or functions for chatbot queries like "explain decision at time t".
8. **Meta-learning hooks**
    - Module to compute rolling performance of multiple agents (like in combining-agents notebook).

- Placeholder meta-learner that can reweight or choose agents, ready to be upgraded later.

Great, those choices give a very clear direction. Below is a tightened "spec for Copilot" incorporating your answers (A/A/A/A/A/B/C/D/D) plus MT5 + FTMO details. Then there are a few extra clarifying questions at the end so we can make this even more precise.

Spec: Self-Healing FTMO RL Trading Bot (Quantra template → upgraded)

1. Core architecture and algorithm

- **Base algorithm**
  - Start with the existing Quantra DQN-style setup: Game environment + two Keras networks (Q and R) as in Model-Solution-Template_-Building-the-RL-Model.ipynb.
  - Maintain the DQN approach as long as it supports FTMO objectives; only switch to PPO later if experiments show DQN cannot meet constraints.
- **Code organization**
  - Keep existing classes and functions (Game, ExperienceReplay, run, reward functions) but move towards a modular structure:
    - env/ftmo_game.py – FTMO-aware Game class (inherits or wraps Quantra Game).
    - agent/dqn_agent.py – init and train DQN with Keras as Quantra does.
    - training/curriculum_trainer.py – curriculum phases + transfer learning.
    - monitoring/ – W&B logging, SHAP explanations, dashboard hooks.
    - execution/mt5_bridge.py – live/forward execution via MetaTrader 5.

---

2. Trading mode & FTMO constraints

- **Mode toggle**
  - Add a config key: TRADING_MODE: "ftmo" | "free".
  - In FTMO mode:

        - Apply FTMO reward and risk constraints.
        - Enforce daily stop when daily loss limit or target hit.
    - In free mode:
        - Use risk rules similar to FTMO but without the hard daily stop (more emphasis on risk-adjusted PnL).
        - Configurable via FREE_MODE_PROFILE if you want to tweak this later.
- **FTMO daily goals (primary objective)**
    - Expose runtime parameters (via config and dashboard):
        - daily_profit_target_pct (default 2.5).
        - daily_max_drawdown_pct (default 1.0).
    - Environment should track, for each "FTMO day":
        - Starting equity.
        - Intraday equity high.
        - Current drawdown from intraday high.
        - Whether daily target has been reached.
- **Strict enforcement with learning**
    - Hard rule in FTMO mode:
        - If current equity ≤ (intraday high × (1 − daily_max_drawdown_pct)), stop trading for that day (no new positions).
        - If equity ≥ (start_equity × (1 + daily_profit_target_pct)), you may:
            - Either stop trading for the day, or
            - Strongly penalize additional risk (configurable).
    - The RL agent still "learns" via reward shaping; the environment enforces the hard boundary.

---

3. Reward design (FTMO mode vs free mode)

Use Quantra's reward structure as a base (pnl functions in Reward-System.ipynb) and extend it.

- **Base PnL reward**
    - Keep rewardexponentialpnl as one available component.
    - Reward at trade close:
        - pnl_pct = percentage PnL including costs as defined by getpnl.
        - reward_base = f(pnl_pct); can be pure PnL, positive PnL, exponential PnL, etc.
- **FTMO-specific reward shaping (priority: pass challenge)**
    - In FTMO mode, compute additional terms:
        - progress = (current_daily_equity - daily_start_equity) / (daily_start_equity * daily_profit_target_pct)
            - Clamp progress between 0 and >1.
        - drawdown = (intraday_high_equity - current_daily_equity) / intraday_high_equity.
    - Example combined reward:
        - reward = reward_base
        - Add positive term proportional to progress when under risk limits.
        - Subtract large penalty if drawdown approaches or exceeds daily_max_drawdown_pct.
        - Add small bonus for *finishing the day* near target without violating drawdown.
    - Keep reward=0 for steps where game/trade isn't finished, like Quantra's design.
- **Free mode reward**
    - Focus on risk-adjusted return:
        - For now, keep exponential PnL or Sharpe-like reward as described in Reward-System.ipynb.

- No hard daily stop, but still penalize large drawdowns.

---

4. Curriculum + transfer learning (4 phases)

Implement 4 explicit curriculum phases.

- **Phases (strict → moderate → FTMO realistic)**
  - Phase 0 – Strict sandbox:
    - Very tight position limits (maybe micro-lots size in MT5).
    - Very conservative daily risk (e.g., 0.25–0.5% max drawdown).
    - Limited hours (e.g., only main session) to simplify dynamics.
  - Phase 1 – Less strict training:
    - Looser constraints, bigger allowed risk, more hours.
  - Phase 2 – FTMO-like conditions:
    - Use the exact FTMO daily rules or close approximation.
  - Phase 3 – Production style FTMO:
    - Same FTMO constraints, but trained on larger data window or new instruments.
- **Implementation**
  - Represent curriculum as a list in a config:
    - Each phase includes:
      - Data slice (dates/instruments).
      - Risk params (max drawdown, target).
      - Reward function choice.
      - Action space and leverage.
  - After each phase:
    - Save network weights (.h5) and replay memory as Quantra already does.
    - In the next phase, load these weights (transfer learning).

- Allow optional layer freezing:
    - Early layers (feature extraction) frozen, later layers fine-tuned.

---

5. Training & Spark usage (fast retraining)

- **Spark scope (your choice C: start with parallel runs)**
    - Do not distribute the internals of a single RL run yet.
    - Use Spark (or Ray, etc.) to:
        - Run multiple training jobs with different seeds / hyperparameters / curriculum tweaks in parallel.
        - Each job:
            - Loads same base code and data.
            - Writes to a unique run directory and W&B run ID.
- **Fast retraining**
    - Ensure training can be restarted quickly by:
        - Reloading latest best weights for each phase.
        - Sampling smaller windows for quick experiments.
        - Using small batch sizes and partial episodes when exploring hyperparameters.

---

6. Dashboard & config interface (Streamlit/Dash)

- **Tech stack (your choice A)**
    - Prefer Streamlit or Plotly Dash for the first dashboard:
        - A single script, e.g. monitoring/dashboard_app.py.
- **Dashboard features**
    - Inputs:
        - TRADING_MODE (toggle).
        - Daily target %, daily max drawdown %.

        - Capital / account size.
    - Displays:
        - Daily PnL and cumulative PnL.
        - Current daily drawdown vs limit.
        - Number of trades, win rate, average R:R.
        - FTMO metrics (e.g. days under drawdown limit, days hitting target) based on backtest log from run and tradeanalytics.

---

7. Explainability: SHAP + chatbot (real-time on demand)

- **SHAP integration (choice B: near real-time when asked)**
    - Wrap the trained DQN model (Keras) with a SHAP explainer:
        - Given a state vector (as assembled in Game/Assemble-States), compute feature contributions to Q-values or selected action.
    - Implement function:
        - explain_action(state, metadata):
            - Runs the network.
            - Computes SHAP values per feature.
            - Returns a summarized explanation (top N features, direction and magnitude).
- **Chatbot hooks**
    - Provide methods usable by a chatbot:
        - “Why did the agent go long/short at time t?” → call explain_action on stored state from trade logs.
        - “What features matter most this week?” → aggregate SHAP values over trades and show in dashboard.
- **Logging**
    - Log SHAP summaries periodically (e.g., per day or episode) to W&B or local files for monitoring.

---

8. Meta-learning hooks (later extension)

- **Initial role (choice D, all of the above, but future)**
    - For now, create:
        - A module that:
            - Reads returns and drawdowns of multiple trained agents (see Combining-the-Agents notebook).
            - Computes rolling Sharpe / drawdown / correlation.
        - A basic capital allocator that can weight or choose agents.
    - Later this can be upgraded to:
        - Hyperparameter tuner (adjust learning rate, exploration, etc.).
        - Curriculum controller (decide when to move to next phase).

---

9. MetaTrader 5 integration

- **Bridge design**
    - Create execution/mt5_bridge.py that:
        - Connects to MT5 (login, symbol, timeframe).
        - Streams live or replayed ticks/bars into the Game environment.
        - Sends actions (buy/sell/flat) as orders, respecting:
            - Position sizing based on account size and risk per trade.
            - FTMO drawdown and daily target rules.
- **Forward testing**
    - Use the same environment but with MT5 prices instead of historical CSV/pickle.
    - The forward test loop should:
        - Run in FTMO or free mode.
        - Log all trades and metrics, same format as backtest.

Here is a clean spec that matches exactly what you asked for and extends it to phases and rewards.

---

1. Indicator set (to be computed on 1D, 1H, 15m, 1m)

From Example Trading Strategies portfolio

- STRAT-001 – Regime Pulse Tracker
    - Bollinger Bands HTF: BB(200), shift 0
    - Bollinger Bands LTF: BB(20), shift 0
    - RSI LTF: RSI(7), no shift
- STRAT-002 – CCI Surge Sentinel
    - CCI fast: CCI(30), no shift
    - CCI slow: CCI(100), no shift
    - RSI exit: RSI(7), no shift
- STRAT-003 – CCI Trinity Vanguard
    - CCI fast: CCI(14), no shift
    - CCI mid: CCI(100), no shift
    - CCI slow: CCI(900), no shift
    - SMA on each CCI:
        - SMA(20), shift 0, applied to CCI(14)
        - SMA(20), shift 0, applied to CCI(100)
        - SMA(20), shift 0, applied to CCI(900)
- STRAT-004 – SMA Stack Prophet
    - SMA slow: SMA(50), shift 0
    - SMA fast: SMA(4), shift 0
    - SMA fast shifted: SMA(4), shift +4
- STRAT-005 – SMA Reversion Rally

    - SMA fast: SMA(30), shift 0
    - SMA slow: SMA(50), shift 0
    - RSI reversion: RSI(5), no shift
- STRAT-006 – Dual Momentum-Volatility Filter
    - ADX: ADX(14) compared to its SMA: SMA(1), shift +5, applied to ADX(14)
    - ATR: ATR(14) compared to its SMA: SMA(1), shift +5, applied to ATR(14)
- STRAT-007 – SMA Fan Accord
    - SMA base: SMA(50), shift 0
    - SMA fan 1: SMA(4), shift 0
    - SMA fan 2: SMA(4), shift +1
    - SMA fan 3: SMA(4), shift +2
    - SMA fan 4: SMA(4), shift +3
    - SMA fan 5: SMA(4), shift +4
- STRAT-008 – CCI BB Outbreak Hunter
    - CCI fast: CCI(30), no shift
    - CCI mid: CCI(100), no shift
    - CCI slow: CCI(300), no shift
    - Bollinger Bands on each CCI: BB(14, deviation 1, shift 0)
- STRAT-009 – Index Open Breakout (“0009”)
    - SMA filter: SMA(200) on 5-minute chart, shift 0
- STRAT-010 – Red News + COT Bias
    - SMA pullback: SMA(200), shift 0
- STRAT-011 – Shifted CCI Momentum Aligner
    - HTF CCI gravity: CCI(140), no shift
    - LTF CCI trigger: CCI(14), no shift
    - SMA on HTF CCI: SMA(1), shift +4, applied to CCI(140)

Extra indicators you specified

(All applied on **all four timeframes: 1D, 1H, 15m, 1m**)

- ATR:
    - ATR(14)
    - SMA(1), shift +2, applied to ATR(14)
- CCI:
    - CCI(30), no shift
    - SMA(1), shift +2, applied to CCI(30)
    - CCI(100), no shift
    - SMA(1), shift +2, applied to CCI(100)
- Bollinger upper bands (for SMA-on-upper logic):
    - For each relevant Bollinger:
        - Upper band of BB(20)
          deviation 1
        - Upper band of BB(200)
          deviation 1
    - On each upper band:
        - SMA(4), shift +8, applied to upper band
        - SMA(4), shift +8, applied to lower band

Here is the tightened version of the phase logic exactly as you described, with no extra behavior added.

Phase 1 – CCI alignment (1m + 15m)

Indicators (per asset, per timeframe 1m and 15m):

- CCI(30)
- SMA(1), shift +2, applied to CCI(30)

- CCI(100)
- SMA(1), shift +2, applied to CCI(100)

Condition to allow NEW trades (must hold on both timeframes):

For each asset:

- On 1m:
    - Either:
        - CCI30_1m > SMA1_shift2_CCI30_1m and CCI100_1m > SMA1_shift2_CCI100_1m
    - Or:
        - CCI30_1m < SMA1_shift2_CCI30_1m and CCI100_1m < SMA1_shift2_CCI100_1m

AND

- On 15m:
    - Either:
        - CCI30_15m > SMA1_shift2_CCI30_15m and CCI100_15m > SMA1_shift2_CCI100_15m
    - Or:
        - CCI30_15m < SMA1_shift2_CCI30_15m and CCI100_15m < SMA1_shift2_CCI100_15m

Trading rules:

- New trades:
    - Only allowed when the above “both above or both below in same direction on both TFs” condition is true.
    - The agent chooses:
        - Buy or sell.
        - Lot size.
- When condition is true:

    - Agent is **allowed** to open a new trade (not forced to always be in, per your correction).
- When condition becomes false:
    - **No new trades allowed**.
    - Existing positions may remain open; the agent must learn **when to close** them on its own.

Phase 2 – SMA on Bollinger upper band (1m + 1H), no BB midlines

Indicators (per asset, per timeframe 1m and 1H):

- Upper band of BB(20) and/or BB(200) (whichever you decide to use as "upper band").
- SMA(4), shift +8, applied to the chosen Bollinger upper band on each TF.

Let's name:

- SMA_upper4_shift8_1m = SMA(4), shift +8, applied to upper band on 1m.
- SMA_upper4_shift8_1H = same on 1H.

Condition to be (and stay) in trade (must hold on both TFs and in same direction):

For each asset:

- On 1m:
    - Either price_1m > SMA_upper4_shift8_1m
    - Or price_1m < SMA_upper4_shift8_1m
- On 1H:
    - Either price_1H > SMA_upper4_shift8_1H
    - Or price_1H < SMA_upper4_shift8_1H

The direction must match on both TFs:

- "Above on both":
    - price_1m > SMA_upper4_shift8_1m AND
      price_1H > SMA_upper4_shift8_1H

- OR “below on both”:
    - price_1m < SMA_upper4_shift8_1m AND
      price_1H < SMA_upper4_shift8_1H

Trading rules:

- When this condition holds (above on both, or below on both):
    - The agent **must be in an active trade** (long or short).
    - If flat when condition becomes true, environment forces entering a trade; agent still decides direction and lot size.
- Outside these conditions:
    - Environment can keep the agent flat (no requirement to be in trade).
    - New trades are **not required**; can be prohibited if you want a pure regime-filter phase.

Phase 3 – Bollinger midlines (1m + 15m)

(As you confirmed is correct.)

Indicators (per asset, per timeframe 1m and 15m):

- BB(200), shift 0 → use **middle line**.
- BB(20), shift 0 → use **middle line**.

Define midlines:

- BB200_mid_1m, BB20_mid_1m.
- BB200_mid_15m, BB20_mid_15m.

Condition to be in trade (must hold on both TFs):

For each asset, on **each timeframe (1m and 15m)**, price must be above or below the middle levels of both BBs:

On 1m:

- Either:
    - price_1m > BB200_mid_1m AND price_1m > BB20_mid_1m

- Or:
    - price_1m < BB200_mid_1m AND price_1m < BB20_mid_1m

On 15m:

- Either:
    - price_15m > BB200_mid_15m AND price_15m > BB20_mid_15m
- Or:
    - price_15m < BB200_mid_15m AND price_15m < BB20_mid_15m

Trading rules:

- When these conditions are jointly met on both 1m and 15m:
    - Agent **must** be in an active trade (long or short).
- Outside these conditions:
    - Agent may be flat (no requirement to hold a position).

Phase 4 – FTMO consistency phase

(As you confirmed is correct.)

Conditions:

- No indicator-based gating (Phase 1–3 filters are all off).
- Agent can open/close trades at any time on any asset/timeframe.

Objectives and constraints:

- Target: reach **+2.5%** of initial balance per day.
- Hard risk: **1% trailing daily max drawdown**:
    - If equity drops 1% below that day's equity high, trading stops for that day, and day is marked as a failure.academy.ftmo

Progressive rewards (unchanged from previous message):

- Daily pass/ok/fail reward.
- Streak bonus for consecutive pass days.

- Extra bonus for low daily DD (well under 1%).ftmo+3

To wire this into code, the next useful step is a small mask_phase1(state), mask_phase2(state), etc. function spec. Do you want to define those as boolean functions like can_open_new_trade_phase1(state, asset) and must_be_in_trade_phase2(state, asset) next?

Here's what you just locked in, translated into a clean "master spec" that Claude can build from.

---

1. State / observation space

1.1 Price & volume

Choice: **1A = A3 (full OHLCV)**

For each asset and each timeframe (1D, 1H, 15m, 1m), include:

- open
- high
- low
- close
- volume
- ATR(14)
- SMA(1), shift +2, applied to ATR(14)

Plus all indicators you listed (BB, CCI + SMAs, SMA fans, ADX + SMA, RSI, etc.), each computed separately on each timeframe.

1.2 Time & calendar

Choice: **1B = B2 (cyclical)**

Use **CET/CE(S)T** as the reference timezone for "end of day" and all time-of-day features. That means:

- Convert MT5 server time to CET/CE(S)T before encoding.

Features:

- sin_hour = sin(2π * hour_CET / 24)
- cos_hour = cos(2π * hour_CET / 24)
- sin_dow = sin(2π * day_of_week / 7) (0=Monday,...)
- cos_dow = cos(2π * day_of_week / 7)

1.3 Position & account context

Choice: **1C = C2 (extended)**

Per asset:

- position_side ∈ {−1, 0, +1}
- position_size_pct_equity (position notional or risk as % of equity)
- unrealized_pnl_pct (unrealized PnL as % of equity)

Account-level:

- equity_pct_change_today
- current_daily_drawdown_pct
- days_in_current_streak (consecutive pass days under FTMO profile)

---

2. Action space

2.1 Direction + discrete size

Choice: **2A = A1 (3 levels each side)**

Per asset, at each decision step (1m):

- flat
- buy_small
- buy_med
- buy_large
- sell_small

- sell_med
- sell_large

The agent selects one action per asset; the network outputs per-asset actions.

2.2 Risk tie-in

Choice: **2B = "RL defines it itself"**

- Environment enforces only **hard account-level constraints** (FTMO profile and any absolute caps you might add later).
- **No fixed mapping** like "small = 0.25% risk"; instead:
    - Claude must define a parameterization where:
        - Each discrete size maps to a fraction of available risk, but those fractions are **learnable / tunable** (e.g., via meta-parameters or a policy head).
- You can explicitly state to Claude:
    - "Implement internal risk fraction parameters for small/med/large that can be optimized (not hard-coded)."

---

3. FTMO profile & risk constraints

3.1 Daily profit & drawdown

Choice: **3A = A1**

Default FTMO-like profile:

- daily_profit_target_pct = 0.025 (2.5% of starting daily equity)
- daily_max_drawdown_pct = 0.01 (1% trailing DD from intraday equity high, including open trades)academy.ftmo

Rules:

- If equity_today <= intraday_high_today * (1 − 0.01) → stop trading for the day, mark day as failure.academy.ftmo
- If equity_today >= start_equity_today * (1 + 0.025) → mark day as target hit; behavior after hit (keep trading vs lock in) can be tuned, but must respect drawdown rule.

3.2 Max positions & exposure

Choice: **3B = "RL defines it itself"**

- No fixed caps like "max 5 positions" in the spec.
- However:
    - Environment must still ensure **basic safety**, e.g., no obviously insane leverage (Claude can set some absolute broker-style limits).
    - Policy is free to learn how many instruments to trade concurrently.

---

4. Curriculum progression

Choice: **4 = A2 (performance-based)**

Advance between phases based on **real FTMO-style performance** in that phase.

- For each phase:
    - Track daily metrics under that phase's masking conditions.
    - When the system achieves **10 consecutive pass days**:
        - Pass day = daily return ≥ 2.5% **and** no violation of 1% trailing daily DD.
    - Then:
        - Move to next phase (1→2→3→4).

No minimum episode count required; phase advancement is purely performance-based.

---

5. MT5 universe & data ranges

5.1 Symbols

Choice: **5A = A2 (mixed FX + indices)**

Default MT5 instruments:

- EURUSD
- GBPUSD
- XAUUSD

- US30
- NAS100

(Claude should parameterize this as a list so you can change it.)

5.2 Time ranges

Choice: **5B = B1**

- **Training window**: 2018-01-01 → 2022-12-31
- **Validation window**: 2023-01-01 → 2023-12-31
- **Forward test window**: 2024-01-01 → present

All aligned to MT5 price history but evaluated in CET/CE(S)T for daily boundaries.

This now gives a pretty complete specification of:

- What the agent sees (state).
- What it can do (actions).
- The risk & FTMO constraints.
- How the curriculum phases advance.
- Where data comes from.

From here, the natural next step is a **single YAML/JSON config schema** that encodes all of this in one place (indicators, phases, FTMO profile, MT5 instruments, etc.). Do you want to do that next (i.e., design the config file structure and keys), or do you want to define the exact **reward formula components** (weights for PnL vs streak bonus vs DD) more concretely first?
